Monitor Remote

A developer's step-by-step build guide

Prepared 1 October 2026 from the current project source.

This guide explains how to recreate and maintain the app as its developer. It presents a reproducible implementation sequence reconstructed from the source and project records, rather than an exact transcript of every historical edit. Code samples marked as excerpts come from the implementation; the source files contain the complete code.

The finished product lets iPhone Safari control external monitor brightness and request all Windows displays off or awake over a trusted local network. A portable Windows launcher starts the server, shows the phone URL and QR code, manages pairing, and stays in the tray.

No cloud service, database, frontend framework, or iPhone native app is required. The PC must remain running and awake. Display off is separate from shutting down the PC, and display wake cannot wake a sleeping PC.

Reading order

1. Define the behavior and architecture.
2. Create the server project and static web interface.
3. Implement native monitor control and isolate it in workers.
4. Add pairing, sessions, API protection, and endpoints.
5. Make the phone interface resilient to rapid input and network loss.
6. Add the Windows launcher and authenticated local management.
7. Add tray behavior, preferences, startup, QR code, and icon.
8. Test, publish, distribute, and maintain the project.

Step 1 - Define the system

Start with two distinct controls: brightness for an individually enumerated physical monitor, and off/wake for all displays. Brightness uses Windows' monitor configuration APIs and depends on the monitor and connection supporting DDC/CI. Power uses Windows display control APIs and does not claim a verified physical power state.

Choose ASP.NET Core Minimal APIs for the local HTTP service, plain HTML/CSS/JavaScript for Safari, and Windows Forms for a convenient desktop launcher. Keep native calls out of the web request process by spawning the server executable in worker mode.

Architecture:

```
iPhone Safari
  | HTTP, session cookie, JSON, LAN port 8765
  v
ASP.NET Core server -> short-lived --worker process
  ^                         | Win32 interop
  | local named pipe        v
Windows Forms launcher   Windows / physical monitors
  |
  + owns server and children through a Windows job
```

A typical brightness request goes through these stages: Safari submits a percentage; the server validates the session and input; a semaphore serializes hardware access; a worker enumerates monitors, maps the percentage into the native range, sets brightness, and reads it back; the server returns JSON; the interface reports the confirmed or unconfirmed result.

Implementation map:

- Program.cs: HTTP hosting, request protection, routes, worker orchestration.
- Hardware.cs: native enumeration, brightness, power, simulated hardware.
- PairingState.cs: pairing code, rate limit, sessions, renewal/revocation.
- LocalManagement.cs and ManagementProtocol.cs: local launcher/server protocol.
- wwwroot/index.html, style.css, app.js: phone controller.
- desktop/Program.cs and MainForm.cs: Windows entry point, window, tray, QR.
- desktop/ServerController.cs and OwnedJob.cs: server lifecycle and ownership.
- desktop/Preferences.cs: local settings and sign-in registration.
- desktop/LauncherChecks.cs and tests/: automated checks.
- package.ps1: portable distribution build.

Step 2 - Set up the projects

Development prerequisites are Windows, the .NET 10 SDK, Node.js with built-in fetch support for the checks, and PowerShell. A compatible external monitor with DDC/CI enabled is needed for real hardware testing. The self-contained published app does not require its user to install the SDK.

To reproduce the initial scaffold in a new empty folder, use:

```
dotnet new web -n MonitorRemote -o . --framework net10.0
dotnet new winforms -n MonitorRemote.Desktop -o desktop --framework net10.0
dotnet add desktop/MonitorRemote.Desktop.csproj package QRCoder --version 1.8.0
```

These are reconstruction commands. Do not run project scaffolding over this existing source tree. Remove or replace the template form when implementing MainForm. Use the checked-in project files as the authority for the finished configuration.

The server uses Microsoft.NET.Sdk.Web, net10.0, implicit usings, and nullable analysis. Since SDK projects include C# files recursively, its project file explicitly excludes desktop/**/*.cs and generated/distribution directories. Otherwise the server would attempt to compile the launcher too. App and docs are excluded from server content to prevent publishing the portable package into itself.

The launcher uses Microsoft.NET.Sdk with OutputType WinExe, net10.0-windows, and UseWindowsForms true. It references QRCoder 1.8.0, embeds its ICO resource, and links ../ManagementProtocol.cs so both processes use the same request/reply types.

For the existing project, begin with:

```
dotnet build MonitorRemote.csproj
dotnet build desktop/MonitorRemote.Desktop.csproj
```

Restore is performed by the build as needed. The desktop dependency must be available from NuGet or the local package cache. Keep the full source separate from App, which is a ready-to-run distribution.

Step 3 - Host the web interface and API

Program.cs first detects --worker and immediately delegates to Hardware.Run. Ordinary startup creates the web application, reads MONITOR_PORT (default 8765), and binds HTTP to 0.0.0.0 so the phone can reach the PC over IPv4.

Source excerpt:

```
if (args.FirstOrDefault() == "--worker") {
    Hardware.Run(args.Skip(1).ToArray()); return;
}
var builder = WebApplication.CreateBuilder(args);
var port = int.TryParse(Environment.GetEnvironmentVariable("MONITOR_PORT"),
    out var p) ? p : 8765;
builder.WebHost.UseUrls($"http://0.0.0.0:{port}");
var app = builder.Build();
```

The middleware is registered before default-file and static-file serving. UseDefaultFiles resolves / to index.html; UseStaticFiles serves the assets in wwwroot. API routes return JSON through Minimal APIs. A console startup prints the six-digit pairing code and discovered IPv4 URLs. The desktop version obtains the code through its local management pipe instead.

Start the source server with dotnet run from the project root. Its browser URL is http://127.0.0.1:8765 locally or http://<PC-private-IP>:8765 on the phone. PC Ethernet and phone Wi-Fi can work together when they share a reachable LAN; they do not need identical connection types.

The server sets Cache-Control: no-store, X-Content-Type-Options: nosniff, and a Content-Security-Policy allowing only its own scripts/styles and blocking framing. Keep JavaScript and styles in external files to fit this policy.

When launched from the desktop, ServerController sets the working directory and passes --contentRoot pointing to the bundled server directory. This makes web assets resolve independently of the directory from which the user double-clicked the executable.

Step 4 - Implement native brightness

Hardware.cs uses P/Invoke rather than a third-party monitor library. It declares EnumDisplayMonitors in user32.dll and the physical monitor functions in dxva2.dll. Its Physical structure contains an IntPtr handle and a fixed 128-character Unicode description, matching the native layout.

First enumerate logical display monitors. For each HMONITOR, call GetNumberOfPhysicalMonitorsFromHMONITOR and GetPhysicalMonitorsFromHMONITOR. Collect the physical handles, then expose their current enumeration index as the API ID. Always call DestroyPhysicalMonitor in finally for each acquired handle, including after failures.

GetMonitorBrightness returns minimum, current, and maximum native values. A monitor is supported only when that read succeeds and maximum exceeds minimum. Convert native brightness to a percentage:

```
percentage = round(100 * (current - minimum) / (maximum - minimum))
target = minimum + round((maximum - minimum) * requestedPercent / 100)
```

For a brightness write, validate the index against the freshly enumerated handles, read the native range, compute target, and call SetMonitorBrightness. Then call GetMonitorBrightness again. confirmed is true only when the readback succeeds and actual equals target. Return nullable brightness when readback is unavailable.

Representative response:

```
{"requested":25,"confirmed":true,"brightness":25}
```

If the monitor rounds differently, return its actual percentage and confirmed false. If the write succeeds but readback fails, return brightness null and confirmed false. Never equate a successful write call with verified physical output.

IDs are enumeration positions, not durable device identities. Unplugging or rearranging displays can change them. Refresh after cable changes before sending more writes. Internal laptop displays and some docks/adapters do not support this method. The current implementation does not use WMI brightness as a fallback.

Step 5 - Implement display off and wake

The power branch runs before physical monitor enumeration because these commands target Windows' display behavior globally.

For off, SendMessageTimeout broadcasts WM_SYSCOMMAND with SC_MONITORPOWER and lParam 2. The source uses HWND_BROADCAST 0xffff, message 0x112, wParam 0xf170, SMTO_ABORTIFHUNG 2, and a 2000 ms timeout. A zero return becomes a worker error.

For wake, call SetThreadExecutionState with ES_DISPLAY_REQUIRED (0x00000002). This wakes the display and resets Windows' display idle timer. The implementation intentionally omits ES_CONTINUOUS: the worker exits and the normal display timeout should apply afterward. The validation history records this change from an earlier wake broadcast.

Both successful operations return:

```
{"requested":true,"confirmed":false}
```

This acknowledges the Windows request, not the visible state of every display. The implementation has no physical power-state readback. Screen savers and lock screens are separate from display power; the app does not unlock the PC or dismiss them.

In the interface, turning displays off asks for confirmation. Both power buttons lock during an operation. The UI explains that the PC remains running and keyboard/mouse recovery may be necessary. Test wake with a recovery method available before relying on phone-only control.

Step 6 - Isolate hardware operations

Native monitor drivers can block. Program.cs therefore launches a separate worker per list, brightness, or power command instead of calling Hardware directly in the HTTP process.

A SemaphoreSlim(1) serializes all hardware operations globally, including monitor reads. A request waits at most two seconds for the gate; failure returns HTTP 503. After acquiring it, the server starts its own executable with --worker and command arguments. When hosted by dotnet, it first passes the assembly path. ArgumentList avoids shell-built command strings.

The worker redirects stdout and stderr, runs with UseShellExecute false and CreateNoWindow true, and inherits the environment. Read both streams asynchronously to avoid pipe-buffer deadlocks. WaitForExitAsync has an eight-second deadline. On timeout, kill the worker tree, wait for exit, and return HTTP 504. A nonzero exit returns 502 with the worker error. Success returns stdout as application/json. Release the gate in finally.

```
HTTP request -> wait for gate (2 s maximum)
             -> start worker -> native operation (8 s maximum)
             -> parse/display JSON response
```

The browser has its own 12-second fetch deadline. A browser timeout does not prove the command failed: it may already have reached the hardware. Reconnect and read the actual brightness before retrying. Hardware writes are not replayed automatically after reconnection.

MONITOR_DEMO=1 switches the worker to simulated responses before native calls. Listing returns a supported Demo monitor at 50%; brightness returns the requested value with no confirmed readback; power returns an unconfirmed simulated acknowledgement. Demo writes are not persisted as a simulated brightness model.

Step 7 - Add pairing and request protection

PairingState stores its data in memory under a lock. It generates a cryptographically random six-digit code from 100000 through 999999, valid for ten minutes. Renewal chooses a different code, resets expiry, and clears the attempt counters.

Pairing tracks attempts by remote IP in a ten-minute window. The first ten attempts are allowed to be evaluated; further attempts return 429. Incorrect or expired codes return 401. Successful pairing creates 32 random bytes encoded as a hex token and stores a twelve-hour server-side expiry.

The server sends session as an HttpOnly, SameSite=Strict cookie with a twelve-hour MaxAge. It is not Secure because the local transport is HTTP. JavaScript does not read the token; same-origin fetch sends the cookie. Logout removes that token. Renewal preserves existing sessions; revocation renews the code and clears every session. Server restart loses all sessions and creates a fresh code.

Middleware accepts Host values matching localhost, loopback IP addresses, or the implemented private IPv4 ranges: 10/8, 192.168/16, and 172.16/12. For every non-GET request it requires both an exact Origin of http://<request-host> and X-Monitor-Request: 1. All /api paths except /api/pair require a valid session, including /api/status.

The Host check validates the requested host; it is not a firewall rule restricting the client's source IP. Firewall scope and network configuration still matter. Binding to all IPv4 interfaces permits LAN access, so use trusted Wi-Fi, Private-profile firewall access, and no router port forwarding. The implementation does not configure the firewall automatically or implement HTTPS.

Sessions and expired attempt entries are not periodically pruned, and there is no distributed rate limiter or persistent identity store. Those are future concerns if the local single-user design grows substantially.

Step 8 - Define the HTTP contract

Every POST uses JSON plus X-Monitor-Request: 1 and the matching Origin. After pairing, include the session cookie. Browser requests provide Origin automatically; a command-line client must supply it explicitly.

- POST /api/pair: {"code":"123456"}; returns {"paired":true} and cookie.
- POST /api/logout: {}; removes the current session and cookie.
- GET /api/status: {"online":true,"simulated":false} in ordinary mode.
- GET /api/monitors: array of id, name, supported, nullable brightness.
- POST /api/monitors/{id}/brightness: {"value":25}; returns requested, confirmed, nullable brightness.
- POST /api/displays/power: {"state":"off"} or {"state":"wake"}; returns requested and confirmed.

Monitor IDs must parse as nonnegative integers. Brightness values must be integers from 0 to 100. Power state must be off or wake. Invalid route inputs return 400; a valid-looking monitor index that no longer exists fails in the worker.

Example browser request, executed from the paired app page:

```
const response = await fetch('/api/monitors/0/brightness', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Monitor-Request': '1'
  },
  body: JSON.stringify({ value: 25 })
});
const result = await response.json();
```

Status meanings: 401 means missing/expired session or bad pairing code; 403 means Host or POST protection rejection; 429 means pairing attempt limit; 503 means hardware controller busy; 504 means worker timeout; 502 means a worker/native operation failed. Some body binding errors are handled directly by ASP.NET Core.

Monitor list example:

```
[{"id":"0","name":"Generic PnP Monitor","supported":true,"brightness":50}]
```

Step 9 - Build the iPhone interface

index.html contains the connection status, pairing form, monitor container, refresh/logout toolbar, and all-display power card. app.js constructs one monitor card for each returned ID, using DOM methods and textContent for monitor names and messages.

Each card has a 0-100 range slider, an output percentage, presets at 25/50/75/100, a status note, and an explicit retry button. Unsupported controls are disabled and show DDC/CI advice. The empty-monitor message appears when enumeration returns no monitors.

style.css uses a dark palette, a centered 640 px maximum layout, wrapping button groups, 48 px touch targets, visible keyboard focus, and safe-area insets through env(safe-area-inset-*). The viewport includes viewport-fit=cover. Status regions use role=status and live announcements; sliders have accessible names and descriptive notes.

Safari can add the page to the home screen. The page includes apple-mobile-web-app-capable, but there is no service worker, offline cache, or full installable PWA implementation. The Windows icon is not currently configured as an apple-touch-icon for the phone.

On startup, load requests status and monitor readings. A valid existing cookie restores the controller; otherwise pairing is required. An incorrect/expired code has explicit messaging. Some current UI strings still refer to the PC console; with the packaged launcher the code is displayed in its window.

Keep the HTML and JavaScript element IDs in sync. Frontend checks execute the actual script against a minimal DOM, so new elements or event behavior may require updates to that fixture.

Step 10 - Handle rapid input and reconnection

app.js keeps cards in a Map, tracks a session epoch, and stores per-card pending value, running state, revision, confirmed reading, desired value, and debounce timer.

Slider input waits 180 ms before sending; preset buttons submit immediately. While a card has an in-flight write, later changes replace its pending value. When the request finishes, only the latest pending value is sent. Revisions prevent an older response from overwriting a newer visible intent. Successful readback updates the last confirmed value even when the displayed intent has moved on.

On failure, discard queued changes, restore the last confirmed reading when available, and show Retry brightness. When readback differs from the request, show both values. When readback is absent, label the request unconfirmed rather than reporting success as a fact.

Refresh and power are disabled while brightness is active or pending. Brightness is disabled during refresh or power. Different monitor cards can submit while another card is running, but the server's global semaphore serializes native work.

api uses AbortController with a 12-second timeout. A session change increments epoch, aborts tracked requests, clears pending commands, and rejects stale responses. Window offline events also invalidate active work. The UI preserves card nodes across refresh to retain slider identity, while removing nodes for missing monitors.

On visibilitychange, a persisted pageshow, or online, load rechecks the session and readings. If hardware work is still active, refresh waits until idle. Manual Reconnect is available after connection failure. There is no automatic replay of hardware commands and no continuous monitor polling loop in the web page.

Step 11 - Add local launcher management

The launcher must read pairing state and stop its own server without exposing administrative actions to the phone network. Use a named pipe restricted to the current Windows user, plus a random per-launch secret.

ServerController generates a GUID-based pipe name and a 32-byte hex secret, then passes MONITOR_MANAGEMENT_PIPE and MONITOR_MANAGEMENT_SECRET through the child's environment. The server begins LocalManagement after HTTP startup; successful status therefore establishes management readiness after hosting has started.

ManagementProtocol.cs is shared by both projects. Requests contain Version, Secret, and Command. Replies contain Version, Running, optional Code/Expires, and optional Error. Each message is UTF-8 JSON terminated by a newline. The reader enforces a 4096-byte bound. Both sides use protocol version 1.

The server creates NamedPipeServerStream with CurrentUserOnly. It compares the supplied secret with CryptographicOperations.FixedTimeEquals. Supported commands are status, renew, revoke, and shutdown. Unauthorized requests return an error and cannot rotate the code. Malformed or disconnected clients are caught so they do not terminate the listener.

The server allows three seconds for one management exchange. The launcher applies a two-second connect/exchange deadline. It validates the reply version and error. shutdown replies first, then stops the ASP.NET application. These management operations have no HTTP equivalents.

Keep this management pipe separate from the desktop activation pipe. The activation pipe only asks an already-running launcher to restore its window; it does not expose pairing state or control the server.

Step 12 - Own and supervise server processes

ServerController locates server/MonitorRemote.exe relative to the desktop executable. It launches without a console and supplies the configured port, management variables, and content root. It drains redirected stdout/stderr without retaining raw server output, which can contain URLs or credentials.

OwnedJob creates a Windows job object and sets JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE (0x2000). Assign the server to that job so closing the job terminates the owned process tree, including workers under normal inheritance. This prevents the launcher from leaving its server behind.

There is a startup race: the child could spawn workers before being assigned to the job. The implementation resolves it with MONITOR_LAUNCH_GATE, a unique temporary file path. The child waits up to ten seconds for that file; the launcher creates it only after job assignment. The launcher deletes its own gate file in finally.

Readiness retries management status within a fifteen-second startup budget. Failed startup calls Stop to clean up ownership. Port conflicts do not cause the launcher to attach to, terminate, or adopt an unrelated server. Retrying an already-running owned server preserves its actual active port even if the next-start preference has changed.

Stop first requests graceful shutdown, waits up to four seconds, then kills the process tree if necessary. Finally dispose the job and process handles and clear the management secret/name. Exited events update the window when the child disappears unexpectedly.

Do not replace this with broad process-name termination: ownership matters when other apps or another console server may be running.

Step 13 - Finish the desktop experience

desktop/Program.cs uses an STA entry point for Windows Forms. A Local mutex named with the current user SID prevents duplicate launchers in that session. A second instance writes to a current-user activation pipe, and the first instance restores its form when its handle exists.

MainForm builds a DPI-aware window with Start/Stop/Restart, adapter selection, phone URL, offline-generated QR code, pairing code/expiry, renew/revoke controls, next-start port, settings, and diagnostics. It enumerates operational non-loopback adapters and offers their private IPv4 addresses. QR uses QRCoder with error correction Q and GetGraphic(6), generated locally without a web service.

A one-second timer updates the pairing countdown and requests management status while idle. A busy flag prevents overlapping launcher operations. Errors clear readiness, show recovery instructions, and enter the bounded diagnostic log. Only the latest 80 diagnostic lines are retained.

Closing the window hides it to NotifyIcon; a one-time explanation is saved. Tray commands restore the window, start/stop the server, open the controller, or Exit. Exit stops the server and disposes the tray icon. Open local controller uses 127.0.0.1 and the active port; the QR uses the selected LAN address.

Preferences are JSON in %LOCALAPPDATA%/MonitorRemote/desktop.json. Fields are AutoStart (default true), CloseExplained, Port (default 8765), and Address. Invalid or unreadable settings fall back to defaults; out-of-range ports are reset.

Opt-in sign-in startup writes a quoted executable path under HKCU/Software/Microsoft/Windows/CurrentVersion/Run, value MonitorRemote. Disabling removes that value. If registry writes fail, the checkbox rolls back and shows a diagnostic.
Moving the app requires updating the startup path by enabling startup from the new location.

Step 14 - Create and embed the Windows icon

The existing icon is a teal monitor and golden sun on a dark navy rounded background. It is created with vector-style drawing operations, not a generated photograph or external icon service.

desktop/assets contains monitor-remote.svg, monitor-remote.png, monitor-remote.ico, and generate-icon.ps1. The PowerShell script independently draws the geometry with System.Drawing; it does not rasterize the SVG. If changing the design, update both the SVG reference and drawing script to keep them consistent.

The script draws at 1024 by 1024, applies antialiasing, resizes with high-quality bicubic interpolation, and encodes PNG frames for 16, 20, 24, 32, 40, 48, 64, 128, and 256 pixels. It constructs the ICO directory and offsets around these nine PNG frames. The 256-pixel frame is also saved as monitor-remote.png.

Regenerate only when intentionally changing the icon:

```
powershell -ExecutionPolicy Bypass -File .\desktop\assets\generate-icon.ps1
```

The desktop project sets ApplicationIcon to assets/monitor-remote.ico for Explorer/executable branding and embeds the file as MonitorRemote.AppIcon. MainForm loads that embedded resource; NotifyIcon reuses the form icon. Rebuild and republish to update the executable; changing only the source ICO does not modify the already-built App folder.

QRCoder is the explicit third-party package. Include THIRD_PARTY_NOTICES.md with distributions and preserve its MIT notice. The self-contained package also contains Microsoft runtime assemblies.

Step 15 - Verify the implementation

Run these checks from the project root. Build in Debug before smoke.mjs because that test starts bin/Debug/net10.0/MonitorRemote.dll explicitly.

```
dotnet build MonitorRemote.csproj
dotnet build desktop/MonitorRemote.Desktop.csproj
node --check wwwroot/app.js
node tests/smoke.mjs
node tests/frontend.mjs
```

The API smoke test creates its own server on port 18765 with MONITOR_DEMO=1. It checks served UI, pairing, unauthorized access, rejected cross-origin requests, brightness/power input validation, simulated worker JSON, and logout. It does not change hardware settings.

frontend.mjs executes the actual app.js in a minimal DOM with controlled fetch/timers. It tests latest intent, readback differences, unconfirmed results, failure/retry, timeout without replay, session changes, serialized power, foreground recovery, card preservation/removal, logout, and pairing. It does not prove Safari layout, browser focus, or VoiceOver behavior.

The packaged desktop supports --check, which runs LauncherChecks without the GUI and forces demo hardware. To rerun against the existing App package:

```
$env:MONITOR_CHECK_REPORT = Join-Path $PWD 'docs\launcher-check-current.txt'
$check = Start-Process -FilePath .\App\MonitorRemote.Desktop.exe `
    -ArgumentList '--check' -WindowStyle Hidden -Wait -PassThru
$check.ExitCode
Get-Content $env:MONITOR_CHECK_REPORT
Remove-Item Env:\MONITOR_CHECK_REPORT
```

A fresh publish must include the server subfolder; running --check against a bare desktop build lacks that bundled executable. Launcher checks cover readiness, static assets, management authentication, pairing renewal/revocation, oversized message recovery, stop, port conflict cleanup, restart revocation, and closing job ownership.

Historical evidence: docs/VALIDATION.md records successful builds, API/frontend checks, and sixteen packaged launcher assertions; tests/launcher-check-results.txt preserves their report. The user has reported the app works. This documentation task does not recertify every runtime or physical behavior.

Step 16 - Publish and distribute

package.ps1 publishes server and launcher in Release for a selected Windows runtime. Default win-x64 is self-contained. It creates a unique folder under artifacts, publishes the server to server/, publishes the launcher into the package root, and copies README.md and THIRD_PARTY_NOTICES.md. A failed publish throws and retains partial output for diagnosis.

```
powershell -ExecutionPolicy Bypass -File .\package.ps1
```

Resulting layout:

```
MonitorRemote-win-x64-<timestamp>-<suffix>/
  MonitorRemote.Desktop.exe
  runtime DLLs and supporting files
  README.md
  THIRD_PARTY_NOTICES.md
  server/
    MonitorRemote.exe
    server/runtime supporting files
    wwwroot/index.html, style.css, app.js
```

Keep the entire package together. The existing App directory contains the current ready-to-use distribution; source edits do not update it automatically. Build a fresh package, verify it, exit the old app, and only then deliberately replace a deployment folder. Do not overwrite a running package or discard existing source edits.

Optional -FrameworkDependent omits bundled runtimes and requires matching .NET 10 ASP.NET Core and Windows Desktop runtimes on the user's PC. Optional -Runtime win-arm64 selects ARM64; the project history does not validate that target. Executables are unsigned; there is no installer, auto-updater, or certificate-signing pipeline.

For a server-only deployment, publish MonitorRemote.csproj separately. It is the console server, not the full desktop/tray package. Launch with a suitable working directory/content root so its web assets resolve.

Step 17 - Connect and troubleshoot

For normal use, double-click App/MonitorRemote.Desktop.exe, leave automatic server start enabled, select the PC's reachable private LAN address, scan its QR using iPhone Camera, open Safari, and enter the displayed code. Keep the PC awake and the package folder intact.

Enable DDC/CI in the monitor's own menu. If brightness is unavailable, test a direct monitor connection instead of a dock/adapter and refresh discovery. Do not treat an unsupported result as an API authentication failure.

If the phone cannot connect, check that the server shows Running, the chosen IP belongs to the reachable home network, the port matches, and the PC is awake. Guest Wi-Fi/client isolation and VPN routing can block access. A router DHCP reservation can stabilize the PC URL.

If Windows requests firewall access, permit Private networks. The app does not modify rules. If a scoped rule is necessary, the README provides this administrator command; use the actual configured port:

```
New-NetFirewallRule -DisplayName 'Monitor Remote Home LAN' `
    -Direction Inbound -Action Allow -Protocol TCP -LocalPort 8765 `
    -Profile Private -RemoteAddress LocalSubnet
```

If a port is occupied, change the next-start port in the launcher and restart. The launcher does not terminate the other process. Renew pairing when the code expires; use Disconnect all phones to revoke sessions. HTTP carries the code and cookie without transport encryption, so keep access on a trusted LAN and do not expose the port publicly.

After connection loss, reconnect and refresh before retrying a command. After cabling changes, refresh before choosing a monitor. For display wake failure, use keyboard/mouse recovery; the command does not wake a sleeping PC.

Manual release checks still matter: Safari touch/layout/VoiceOver, QR scanning, rapid slider input, Wi-Fi recovery, repeated off/wake, tray and second-instance behavior, sign-in enable/disable, process-crash cleanup including workers, and a target Windows machine without .NET preinstalled.

Step 18 - Maintain the app confidently

To change a feature, trace it through the ownership boundaries. A new phone operation typically needs a route and validation in Program.cs, worker behavior in Hardware.cs, interface changes in app.js/index.html, and demo/test coverage. A new desktop management action needs the shared protocol, LocalManagement, ServerController, and MainForm.

Keep requested values distinct from confirmed readback. Preserve worker isolation, deadlines, physical-handle cleanup, and owned-process cleanup. Keep session epochs and card revisions when changing asynchronous UI logic. Never automatically replay a potentially executed hardware command after a timeout.

Change authentication in PairingState and middleware together; add tests for expiry/revocation and rejected requests. Change packaging through package.ps1 and project exclusions so nested builds do not consume distribution files. For icon updates, regenerate assets and republish both the executable branding and embedded resource.

The reproducible development loop is: edit source, build both projects, run the targeted API/frontend checks, publish a new package, run its launcher --check, then manually exercise the behaviors changed on Windows and iPhone. Preserve the old working distribution until the new package is verified.

Current boundaries: HTTP only; in-memory sessions; enumeration-based monitor IDs; external DDC/CI brightness only; global unconfirmed display power; no PC wake-from-sleep, cloud access, hardware command replay, or automatic updates. These are properties of the implementation rather than promises of missing features.

Source references and development milestones

README.md is the current run/build reference. docs/IMPLEMENTATION.md records the original approach; docs/POLISH_IMPLEMENTATION.md records interface/reliability planning; docs/WINDOWS_APP_IMPLEMENTATION.md records launcher planning. Plans can contain deferred ideas; source and current validation establish implemented behavior.

The recorded progression was a console-hosted local web controller, a native wake correction, phone-interface reliability polish, a portable Windows launcher, a Windows icon, and distribution cleanup into App. docs/VALIDATION.md records milestone checks and their limits. This guide organizes that work into a build sequence a developer can follow and explain.

An editable Markdown copy accompanies the PDF. docs/build-developer-guide.mjs generates the PDF with Node.js alone, so the documentation can be rebuilt without modifying the app or installing a document tool.